

به نام خدا

سامانه‌ی مدیریت آموزش  
پروژه‌ی درس مهندسی نرم افزار

گروه‌یک

اعضا:

عطیه جمشیدپور ۹۴۱۰۳۸۳۵

فاطمه عباسپور ۹۵۱۰۹۶۶۱

عطیه غفارلوی مقدم ۹۵۱۰۵۷۳۲

اسرا کاشانی نیا ۹۵۱۰۵۸۱۶

مریم مرادی شبستری ۹۵۱۰۵۸۶۲

استاد: دکتر سیدهادی سجادی

نیمسال اول ۹۹۹۸

|                                                       |    |
|-------------------------------------------------------|----|
| گام اول.....                                          | ۳  |
| انتخاب ابزار ایجاد پروتوتایپ.....                     | ۳  |
| پروتوتایپ و تطبیق با نیازمندی‌ها.....                 | ۳  |
| داستان‌های کاربر بر اساس صفحات نظیرشان.....           | ۴  |
| گام دوم:.....                                         | ۱۰ |
| قابلیت همکاری.....                                    | ۱۰ |
| فراهم بودن:.....                                      | ۱۰ |
| قابل استفاده بودن:.....                               | ۱۱ |
| امنیت:.....                                           | ۱۲ |
| گام سوم.....                                          | ۱۳ |
| توصیف راهبرد و کلیت سامانه.....                       | ۱۳ |
| معماری سامانه مدیریت آموزش.....                       | ۱۵ |
| تکنولوژی‌های پیشنهادی برای سامانه‌ی مدیریت آموزش..... | ۱۷ |
| تکنولوژی‌های سمت کلاینت.....                          | ۱۷ |
| تکنولوژی‌های سمت سرور.....                            | ۱۷ |
| برنامه‌ی تست نرم افزار.....                           | ۱۸ |
| برنامه آزمون یا Test plan چیست؟.....                  | ۱۸ |
| بخش دوم: تشخیص testing type.....                      | ۱۹ |
| منابع.....                                            | ۲۴ |

## گام اول

انتخاب ابزار ایجاد پروتوتایپ

به دلایل زیر از ابزار moqups استفاده کردیم [۱].

۱. استفاده آسان به علت رابط کاربری مناسب
۲. بدون نیاز به پرداخت هزینه
۳. Feature های مناسب
۴. امکان استفاده به صورت آنلاین
۵. ورود به سایت آسان به کمک اکانت گوگل یا فیسبوک بدون نیاز به ایمیل خاص

پروتوتایپ و تطبیق با نیازمندی‌ها

در راستای برآورده کردن نیاز کاربران نیاز به ایجاد هفده صفحه بود که در زیر آمده اند.

۱. صفحه بانک سوالات که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/bank_of_questions` آمده است.
۲. صفحه درس از دید دانش‌آموزان که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/course_page` آمده است.
۳. صفحه داشبورد سوالات که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/dashboard` آمده است.
۴. صفحه بحث و گفت و گو که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/discussion` آمده است.
۵. صفحه ویرایش محتوای درس که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/edit_lesson` آمده است.
۶. صفحه ورود که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/login` آمده است.
۷. صفحه ایجاد تمرین که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/make_exercise` آمده است.
۸. صفحه ایجاد درس که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/make_lesson` آمده است.
۹. صفحه ایجاد آزمونک که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/make_quiz` آمده است.
۱۰. صفحه پیام‌ها که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/messages` آمده است.
۱۱. صفحه ثبت نام که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/register` آمده است.
۱۲. صفحه یادآور که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/reminder` آمده است.

۱۳. صفحه ارسال پاسخ که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/ send_answer` آمده است.
۱۴. صفحه گزارش نمرات دانش‌آموز که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/ student_score_report` آمده است.
۱۵. صفحه نظرسنجی که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/ survey` آمده است.
۱۶. صفحه گزارش‌گیری و ویرایش نمرات توسط استاد که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/ teacher_edit_score_report` آمده است.
۱۷. صفحه ایجاد بحث و نظرسنجی که پروتوتایپ آن در ضمیمه این سند در `attachments/step1/pages/ make_survey_discussion` آمده است.

در بخش بعد داستان‌های کاربر که در فاز قبل پروژه معرفی شدند آمده‌اند. که در مقابل هر یک صفحات نظیرشان ارجاع داده شده است.

#### داستان‌های کاربر بر اساس صفحات نظیرشان

- (۱) نیازمندی دعوت و مدیریت افراد عضو در کلاس (برای مدرسان)
- توانایی اضافه کردن دانشجویان به کلاس (استاد قادر باشد که یک گروه<sup>۱</sup> از دانشجویان با ویژگی مشترک مشخص (مثلاً افراد یک دوره) را به کلاس اضافه کند)
    - صفحات ۸ و ۵ که به ترتیب به صفحات ایجاد و ویرایش درس اشاره دارند.
  - توانایی حذف افراد از کلاس (آنها دیگر نتوانند صفحه‌ی درس را ببینند)
    - صفحات ۸ و ۵ که به ترتیب به صفحات ایجاد و ویرایش درس اشاره دارند.
- (۲) نیازمندی شرکت در کلاس (برای دانش‌پذیران)
- ایجاد حساب کاربری (ایمیل، رمز عبور، نام کاربری)
    - صفحه ۱۱ که به صفحه ثبت نام اشاره دارد.
  - Login و log out کردن
    - صفحه ۶ که به صفحه ورود اشاره دارد و نوبار بالای تمام صفحات برای خروج
- (۳) نیازمندی به اشتراک‌گذاری محتواهای درسی (برای مدرسان)
- توانایی اضافه کردن درس
    - نوبار بالای صفحات، در بخش درس‌ها و صفحه ۸ که به صفحه ایجاد درس اشاره دارد.

---

<sup>۱</sup> cohort

- توانایی اضافه کردن مباحث جدید به درس (یعنی ظاهر درس به صورت درختی و سلسله‌مراتبی می‌شود که از چند مبحث تشکیل شده -به ترتیب زمان تدریسشان- و هر مبحثی منبع مطالعه، تمرین، کوئیز، نظرسنجی می‌تواند داشته باشد)
    - صفحه ۵ که به صفحه ویرایش درس اشاره دارد.
  - توانایی اضافه کردن منبع درسی (فایل pdf, powerpoint, word و...، عکس، فایل صوتی و ویدیو)
    - صفحه ۵ که به صفحه ویرایش درس اشاره دارد.
  - توانایی پنهان کردن یکی از زیرمجموعه‌های یک مبحث (منبع، نظرسنجی، صفحه‌ی اینترنتی که خود مدرس (یا مدرسان) در سایت ساخته‌اند و...) از دانشجویان
    - صفحه ۵ که به صفحه ویرایش درس اشاره دارد.
  - توانایی قرار دادن پیوند به سایر سایت‌ها در هر یک از مباحث
    - صفحه ۵ که به صفحه ویرایش درس اشاره دارد.
  - توانایی اضافه کردن صفحه‌ی لغات کاربردی<sup>۲</sup> به صفحه درس (به طوری که دانشجویان پیوند آن را پایین صفحه درس ببینند و استاد بتواند لغت جدید اضافه کند یا معنی لغات را ویرایش کند)
    - صفحه ۵ که به صفحه ویرایش درس اشاره دارد. فرض بر این است که ترجمه لغت بازنویسی می‌شود.
- ۴) نیازمندی دسترسی به و استفاده از محتواهای درسی (برای دانش‌پذیران)
- مشاهده دروسی که یک فرد عضو شده در dashboard حساب کاربری آن شخص
    - صفحه ۳ که به صفحه داشبورد درس اشاره دارد.
  - توانایی مشاهده‌ی صفحه‌ی درس و ورود به پیوندهای موجود در آن
    - صفحه ۲ که به صفحه درس اشاره دارد.
  - توانایی مشاهده‌ی ریزنمرات یک دانشجو توسط خودش
    - صفحه ۱۴ که به صفحه گزارش نمرات دانشجو اشاره دارد.
- ۵) نیازمندی ارزیابی و نمره‌دهی به دانش‌پذیران (برای مدرسان)
- توانایی مشاهده‌ی درصد پیشرفت هر یک از دانشجویان در یک درس (چند درصد از کل مشارکت درسی‌اش اعم از تمرین و کوئیز و مشارکت در بحث‌ها و نظرسنجی‌ها در طول

---

<sup>۲</sup> glossary

ترم انجام داده، با توجه به درصد نمره‌ای که استاد به هر یک از این موارد اختصاص داده است.)

- صفحه ۱۶ که به صفحه گزارش‌گیری و ویرایش ارزیابی عملکرد دانشجویان توسط استاد اشاره دارد.
- توانایی اضافه کردن صفحه آپلود تمرین به یک مبحث درس
  - صفحه ۵ که به صفحه ویرایش درس اشاره دارد. با ایجاد تمرین، بخش بارگذاری نیز فعال می‌شود.
- توانایی گذاشتن محدودیت حجم آپلود فایل برای تمرین
  - صفحه ۷ که به صفحه ایجاد تمرین اشاره دارد.
- توانایی مشاهده اعلان در داشبورد استاد وقتی دانشجویی جواب تمرینش را آپلود می‌کند.
  - نوبار در بالای تمام صفحات، در قسمت پیام‌ها.
- توانایی قرار دادن مقیاس برای نمره‌دهی تمارین
  - صفحه ۷ که به صفحه ایجاد تمارین اشاره دارد.
- توانایی اضافه کردن کوئیز به هر مبحث
  - صفحه ۵ که به صفحه ویرایش درس اشاره دارد.
- توانایی اضافه کردن سوال تشریحی به کوئیز
  - صفحه ۱ و صفحه ۹ که به صفحه بانک سوال و ایجاد آزمونک اشاره دارند.
- توانایی اضافه کردن سوال چندگزینه‌ای به کوئیز
  - صفحه ۱ و صفحه ۹ که به صفحه بانک سوال و ایجاد آزمونک اشاره دارند.
- توانایی اضافه کردن سوال صحیح یا غلط به کوئیز
  - صفحه ۱ و صفحه ۹ که به صفحه بانک سوال و ایجاد آزمونک اشاره دارند.
- توانایی قرار دادن محدودیت زمانی برای حل کوئیز
  - صفحه ۹ که به صفحه ایجاد آزمونک اشاره دارد.
- توانایی قرار دادن بارم برای سوالات کوئیز
  - صفحه ۹ که به صفحه ایجاد آزمونک اشاره دارد.
- توانایی ذخیره کردن سوال کوئیز (یک بانک سوالات وجود داشته باشد که در هر درسی هر سوالی که ایجاد می‌شود در آن ذخیره شود تا اگر زمان دیگری خود طراح یا بقیه نیاز داشتند از بانک سوالات سوال بردارند و در کوئیز خود بگذارند)

- صفحه ۱ که به صفحه بانک سوالات اشاره دارد.
- توانایی جستجو در بانک سوالات بر اساس کلمه
  - صفحه ۱ که به صفحه بانک سوالات اشاره دارد.
- توانایی ذخیره کردن نمرات (یک صفحه نمرات باشد که یک جدول داشته باشد که سطرها نام دانشجویان و ستون‌ها هر کدام یک کوییز یا تمرین یا نظرسنجی‌ای که شرکت در آن نمره دارد باشد)
  - صفحه ۱۶ که به صفحه گزارش‌گیری و ویرایش عملکرد دانشجو توسط استاد اشاره دارد
- توانایی مشاهده‌ی نمرات یک دانشجوی خاص توسط استاد یا دستیاران
  - صفحه ۱۶ که به صفحه گزارش‌گیری و ویرایش عملکرد دانشجو توسط استاد اشاره دارد.
- ۶) نیازمندی شرکت در ارزیابی‌ها و تحویل جواب‌ها (برای دانش‌پذیران)
  - توانایی مشاهده جدول زمانی ددلاین تکالیف و زمان کوییزها (از نزدیک‌ترین رویداد به بعد) در داشبورد حساب کاربری
    - صفحه ۳ که به صفحه داشبورد اشاره دارد.
- ۷) نیازمندی گرفتن بازخورد از دانش‌پذیران (برای مدرسان)
  - توانایی اضافه کردن بحث<sup>۳</sup> به هر کدام از مباحث درس (یعنی استاد یک صفحه ایجاد می‌کند و موردی را در آن اعلام می‌کند یا می‌پرسد و دانشجویان در آن صفحه ذیل نوشته‌ی استاد نظرشان در مورد آن موضوع را می‌نویسند. پیوند به این صفحه باید در صفحه اصلی درس ذیل یکی از مباحث موجود باشد)
    - صفحه ۵ که به صفحه ویرایش درس اشاره دارد.
  - اعلان<sup>۴</sup> فرستادن به افراد کلاس هنگام ایجاد بحث (افراد که استاد هنگام ایجاد بحث انتخاب می‌کند در داشبورد حساب کاربری‌شان یا در گوشه صفحه‌شان ای اعلان را ببینند)
    - صفحه ۱۷ که به صفحه ایجاد بحث و نظرسنجی اشاره دارد.
  - توانایی اضافه کردن نظرسنجی به صورت سوال چند گزینه‌ای (که پیوند به آن روی صفحه‌ی درس قرار داده می‌شود)

---

<sup>۳</sup> forum

<sup>۴</sup> notification

○ صفحات ۱۷ و ۲ که به صفحه ایجاد نظر سنجی و بحث و صفحه درس اشاره دارند.

- توانایی مشاهده نتایج نظرسنجی (استاد درصد رای هرکدام از گزینه‌ها را بتواند مشاهده کند)

○ صفحه ۱۵ که به صفحه نظرسنجی اشاره دارد.

۸) نیازمندی دادن بازخورد و شرکت در تصمیم‌گیری‌های مربوط به درس (برای دانش‌آموزان)

- توانایی نظر گذاشتن در ذیل بحث

○ صفحه ۴ که به صفحه بحث و گفت و گو اشاره دارد.

- توانایی انتخاب یکی از گزینه‌های نظرسنجی

○ صفحات ۲ و ۱۵ که به صفحات درس و نظرسنجی اشاره دارد.

۹) نیازمندی مشاهده فعالیت افراد عضو در کلاس (برای مدرسان)

- توانایی مشاهده فعالیت داشتن یا نداشتن دانشجویان در بحث‌ها و نظرسنجی‌ها (جدولی که سطرهای آن نام‌های دانشجویان و ستون‌های آن نظرسنجی‌ها و بحث‌ها هستند که در هر کدام از خانه‌ها اگر دانشجو در نظر گذاشته باشد یا در نظرسنجی شرکت کرده باشد "بله" و در غیر این صورت "خیر" نوشته شده است).

○ صفحه ۱۶ که به صفحه گزارش‌گیری و ویرایش نمرات توسط استاد اشاره دارد.

۱۰) نیازمندی برنامه‌ریزی زمانی و محتوایی برای ارائه درس (برای مدرسان)

- توانایی قرار دادن دلایل برای تمرین‌ها (دانشجویان باید بتوانند تاریخ آن را در صفحه‌ی تمرین ببینند)

○ صفحات ۷ و ۱۳ که به صفحات ایجاد تمرین و تحویل پاسخ اشاره دارند.

- توانایی قرار دادن تاریخ معین برای کوییزها

○ صفحه ۹ که به صفحه ایجاد آزمونک اشاره دارد.

- توانایی قراردادن یادآور<sup>۵</sup> برای تصحیح کوییزها (در یک روز که استاد هنگام طرح کوییز مشخص کند در صفحه داشبورد استاد اعلان یادآوری تصحیح کوییز دیده شود).

○ صفحات ۳ و ۱۲ که به صفحات داشبورد و یادآور اشاره دارد.

۱۱) نیازمندی تبادل پیام (برای مدرسان و برای دانش‌پذیران)

- ایجاد پیام (باید در inbox گیرنده دیده شود)

---

<sup>۵</sup> reminder



- صفحه ۵ که به صفحه پیام‌ها اشاره دارد.
- مشاهده‌ی پیام‌های دریافتی به صورت لیست و هرکدام به صورت تکی
  - صفحه ۵ که به صفحه پیام‌ها اشاره دارد.
- مشاهده پیام‌های خوانده‌نشده با رنگ جداگانه
  - صفحه ۵ که به صفحه پیام‌ها اشاره دارد.

## گام دوم: قابلیت همکاری<sup>۶</sup>:

دلیل:

LMS باید قابلیت همکاری را فراهم کند که از سیستم شناسایی منحصر به فرد کارمندان سیستم (LEE) برای ردیابی دانش آموزان در سراسر سیستم LMS و تبادل داده استفاده کند (جهت ارزیابی فعالیت‌ها- مثلا شرکت در رای گیری‌های کلاسی). وقتی اطلاعات دانش آموز به درستی بارگذاری نشود، سیستم باید گزارش خطا ایجاد کند. وقتی اطلاعات دانش آموز به درستی بارگذاری نشود، سیستم باید گزارش خطا ایجاد کند. علاوه بر این باید توانایی آپلود کردن انواع مختلف فایل وجود داشته باشد (برای تحویل تمرین و انتشار درسنامه‌ها) تاکتیک‌ها:

سه تا از تاکتیک‌ها که دوتا مربوط به مدیریت واسط‌ها و یکی مربوط به مکان‌یابی هستند: (۱) Orchestrate: یک تاکتیک است که از یک مکانیزم کنترل برای هماهنگی و مدیریت استفاده از خدمات خاص که ممکن است نسبت به یکدیگر بی خبر باشند، استفاده می کند. موتورهای گردش کار<sup>۷</sup> نمونه ای از استفاده از تاکتیک Orchestrate است. الگوی طراحی mediator نمونه ای از تاکتیک Tailor Interface است.

(۲) Tailor Interface: تاکتیکی است که قابلیت ها را به یک رابط اضافه یا حذف می کند. قابلیت هایی مانند ترجمه، اضافه کردن بافر یا داده های هموار را می توان اضافه کرد. ممکن است قابلیت ها حذف شوند. نمونه ای از این بین بردن قابلیت ها، پنهان کردن عملکردهای خاص از کاربران غیرمعتبر است. الگوی دکوراتور نمونه ای از تاکتیک Tailor Interface است.

(۳) Discover Service: از این سیستم هنگامی استفاده می شود که سیستم هایی که با هم همکاری دارند باید در زمان اجرا کشف شوند. با جستجو در یک سرویس فهرست شناخته شده، یک سرویس را پیدا کنید. ممکن است در این فرآیند مکان چندین سطح مختلف وجود داشته باشد یعنی یک مکان به مکان دیگری اشاره کند که به نوبه خود می تواند برای سرویس موردنظرمان مورد جستجو قرار گیرد. این سرویس را می توان براساس نوع سرویس، نام، محل، یا برخی ویژگی های دیگر در آن قرار داد.

فراهم بودن<sup>۸</sup>:

دلیل:

چون دانش آموزان هر روزی ممکن است ددلاین تمرین یا امتحان داشته باشند، زمان قطع سیستم باید حداقل از ۲۴ ساعت قبل اطلاع رسانی شود. به روزرسانی های برنامه ریزی شده،

---

<sup>۶</sup> interoperability

<sup>۷</sup> workflow

<sup>۸</sup> availability

وصله‌ها<sup>۹</sup> و پشتیبانی باید بدون اختلال در سرویس رخ دهد. همچنین خرابی یک سرویس خاص نباید کل سیستم را از دسترس خارج کند. تاکتیک‌ها:

- ۱) رأی‌گیری: فرایندهایی که روی پردازنده‌های اضافی اجرا می‌شوند، هر یک از آنها ورودی یکسانی را دریافت می‌کنند و یک مقدار خروجی ساده را که برای رأی دهنده ارسال می‌شود محاسبه می‌کنند. اگر رأی دهنده رفتار غیرمعمول را از یک پردازنده واحد تشخیص دهد، آن را fail می‌کند. الگوریتم رأی‌گیری می‌تواند "رای با اکثریت" یا "مؤلفه مرجع" یا یک الگوریتم دیگر باشد. این روش برای تصحیح عملکرد نادرست الگوریتم‌ها یا خرابی پردازنده استفاده می‌شود و اغلب در سیستم‌های کنترل استفاده می‌شود. اگر همه پردازنده‌ها از همان الگوریتم‌ها استفاده کنند، افزونگی فقط خطای پردازنده را تشخیص می‌دهد و نه خطای الگوریتم. بنابراین، اگر نتیجه یک شکست، شدید باشد، مانند از بین رفتن احتمالی کل thread، اجزای اضافی می‌توانند متنوع باشند.
- ۲) ناظر فرآیند<sup>۱۰</sup>: هنگامی که خطایی در فرآیندی تشخیص داده شود، یک فرآیند نظارت می‌تواند آن فرآیند را حذف کرده و نمونه جدیدی از آن را ایجاد کند که حالت اولیه مناسب داشته باشد.

قابل استفاده بودن:

دلیل:

این سیستم باید بتواند به طور سیار مورد استفاده قرار گیرد (به عنوان مثال، تلفن همراه و تبلت) تا دانش‌آموزان بتوانند مثلاً در راه درس بخوانند و اگر مثلاً در قطار هستند بتوانند تمرین خود را تحویل دهند. این سیستم باید از تمامی نسخه‌های فعلی و نسخه‌های قبلی مرورگرهای وب مدرن از جمله Firefox، Chrome و Safari پشتیبانی کند. سیستم باید از نوشتن زیرنویس فیلم برای آموزش مبتنی بر رایانه و با ویدیو پشتیبانی کند.

تاکتیک‌ها:

دو نوع تاکتیک از این نیازمندی پشتیبانی می‌کند، هر یک برای دو دسته "کاربران" در نظر گرفته شده است. دسته اول، زمان اجرا، شامل مواردی است که در هنگام اجرای سیستم از کاربر پشتیبانی می‌کند. دسته دوم مبتنی بر ماهیت تکراری در طراحی رابط کاربر است و به توسعه دهنده رابط در زمان طراحی کمک می‌کند.

نوع اول:

- ۱) یک الگوی کار را حفظ کنید. در این حالت، مدل حفظ شده، مدل وظیفه<sup>۱۱</sup> است. از الگوی وظیفه برای تعیین محتوا استفاده می‌شود تا سیستم بتواند ایده‌ای از آنچه کاربر تلاش برای انجام آن دارد بدست آورد انواع مختلفی از کمک را ارائه می‌دهد. به

---

<sup>۹</sup> patch

<sup>۱۰</sup> Process monitor

<sup>۱۱</sup> task

عنوان مثال، دانستن این که جملات معمولاً با حروف بزرگ شروع می شوند ، به یک برنامه اجازه می دهد تا حروف کوچک را در آن موقعیت تصحیح کند.

۲) یک مدل از کاربر را حفظ کنید. در این حالت ، مدل حفظ شده مدل کاربر است. این کار دانش کاربر در مورد سیستم ، رفتار کاربر از نظر زمان پاسخ انتظار مورد انتظار و سایر جنبه های خاص برای کاربر یا یک گروه از کاربران را تعیین می کند. به عنوان مثال، حفظ یک مدل کاربری به سیستم اجازه می دهد تا سرعت پیمایش<sup>۱۲</sup> را تنظیم کند تا صفحات سریعتر از آنچه خوانده می شوند حرکت نکنند.

۳) یک مدل از سیستم را حفظ کنید. در این حالت، مدل حفظ شده مدل سیستم است. این رفتار سیستم مورد انتظار را تعیین می کند تا بازخورد مناسبی به کاربر داده شود. مدل سیستم مواردی مانند زمان مورد نیاز برای انجام فعالیت فعلی را پیش بینی می کند.

نوع دوم:

این تاکتیک نوع اصلاح شده ای از تاکتیک انسجام معنایی برای نیازمندی قابلیت تغییر<sup>۱۳</sup> است. رابط کاربری را از بقیه برنامه جدا کنید. منطق انسجام معنایی، محلی سازی تغییرات مورد انتظار است. از آنجا که انتظار می رود رابط کاربری هم در حین توسعه کد و هم بعد از استقرار آن تغییر کند، حفظ کد رابط کاربری به طور جداگانه باعث جلوگیری از اثرگذاری تغییر در آن از تغییر در بقیه ی کد می شود. الگوهای معماری نرم افزار توسعه یافته برای اجرای این تاکتیک و پشتیبانی از اصلاح رابط کاربری عبارتند از:

- مدل-نمایش-کنترل (MVC)

- ارائه-انتزاع-کنترل (PAC)

- Seeheim

امنیت:

دلیل:

چون اطلاعات شخصی کاربران مثل نام و شماره ملی و شغل و سن و حتی بعضا اطلاعات کارت بانکی برای درس های آنلاین در سامانه نگهداری می شود باید امنیت اطلاعات تضمین شود تا از سو استفاده جلوگیری شود.

تاکتیک ها:

۱) تأیید اعتبار کاربران: احراز هویت اطمینان حاصل می کند که کاربر یا کامپیوتر از راه دور در واقع کسی است که ادعا می کند. گذرواژه ها ، گذرواژه های یک بار مصرف ، گواهی های دیجیتالی و شناسایی بیومتریک تأیید هویت را انجام می دهند.

۲) به کاربران مجوز دهید. مجوز تضمین می کند که کاربر معتبر حق دسترسی و تغییر داده یا خدمات را دارد. این کار معمولاً با ارائه برخی از الگوهای کنترل دسترسی در یک سیستم مدیریت می شود. کنترل دسترسی می تواند توسط کاربر یا توسط گروهی از کاربران باشد.

---

<sup>۱۲</sup> scrolling

<sup>۱۳</sup> modifiability

کلاس های کاربران را می توان توسط گروه های کاربر، نقش کاربر یا لیست افراد مشخص کرد.

(۳) محرمانه بودن داده ها را حفظ کنید: داده ها باید از دسترسی غیرمجاز محافظت شوند. محرمانه بودن معمولاً با استفاده از نوعی رمزگذاری در داده ها و لینک های ارتباطی حاصل می شود. رمزگذاری تنها محافظت برای انتقال داده ها از طریق پیوندهای ارتباط عمومی است. این لینک ها می تواند توسط یک شبکه خصوصی مجازی (VPN) یا توسط یک لایه امن سوکت (SSL) برای یک لینک مبتنی بر وب پیاده سازی شود. رمزگذاری می تواند متقارن باشد (هر دو طرف از یک کلید استفاده می کنند) یا نامتقارن (کلیدهای عمومی و خصوصی).

(۴) حفظ صحت<sup>۱۴</sup>. داده ها باید همانطور که مقرر بوده تحویل داده شوند. می تواند اطلاعات اضافی رمزگذاری شده در آن را داشته باشد ، مانند نتایج چک<sup>۱۵</sup> یا نتایج هش، که می تواند به صورت مستقل از داده های اصلی رمزگذاری شوند.

## گام سوم

### توصیف راهبرد و کلیت سامانه

در این بخش برای مشخص کردن معماری یک سامانه ی مدیریت آموزش از راهبرد مبتنی بر مولفه<sup>۱۶</sup> استفاده میکنیم. راهبرد مبتنی بر مولفه یکی از راهبرد های محبوب مهندسی نرم افزار است که در این راهبرد فرض میکنیم که سیستم های نرم افزاری از مولفه های قابل استفاده ی مجدد<sup>۱۷</sup> ایجاد میشوند. این مولفه های در یک مخزن<sup>۱۸</sup> ذخیره میشوند و برای توسعه ی بخش های جدید و سیستم های جدید به کار گرفته میشوند. این راهبرد را به این جهت انتخاب کردیم که به خوبی قابل پیاده سازی روی سیستم های مدیریت آموزش است.

---

<sup>۱۴</sup> integrity

<sup>۱۵</sup> checksums

<sup>۱۶</sup> component-based

<sup>۱۷</sup> reusable components or RC

<sup>۱۸</sup> repository

هر درس یادگیری الکترونیک<sup>۱۹</sup> یک مجموعه از اشیاء یادگیری<sup>۲۰</sup> است که با استفاده از تکنولوژی های مدرن کامپیوتری پیاده سازی میشود و در یک سناریو که برای سازماندهی به فرآیند یادگیری یک از دانش آموزان تعریف شده است، مورد استفاده قرار میگیرد. بنابراین این اشیاء یادگیری یکی از همان مولفه های قابل استفاده ی مجدد هستند.

اشیاء یادگیری دو دسته هستند:

۱. اطلاعات و محتوای اشیاء یادگیری<sup>۲۱</sup>: مباحث تئوری یک درس را مطرح میکند مانند بخش های مختلف و مثال ها.

۲. وظایف اشیاء یادگیری<sup>۲۲</sup>: برای کنترل دانش و همچنین تکلیف دهی در مدت زمان استفاده از درس.

اشیاء یادگیری ساختاری پیچیده دارند. علاوه بر نوع بسته به کاربرد بخش های مختلفی را در بر میگیرند. کاری که وظایف اشیاء یادگیری (LOT) انجام میدهد در واقع ارزیابی محتوای ارائه شده در اطلاعات اشیاء یادگیری (LOI) است. خود وظایف یادگیری اشیاء (LOT) نیز به دو بخش تقسیم میشود: وظایف فردی و وظایف کلی. وظایف کلی برای تمامی دانش آموزان یکسان است اما وظایف فردی برای هر فرد و به طور خاص برای همان فرد تعیین شده است. هر LOT بسته به کاربرد چندین نوع دارد که بخش های متناسب با هدف آن شی یادگیری را پیاده سازی میکند.

علاوه بر پیچیدگی های اشیاء یادگیری یک لیست از فرا داده ها هم باید هنگام پیاده سازی دروس الکترونیک در نظر گرفته شوند. لیست این فرا داده ها در استاندارد های IEEE قابل دسترس است. بدیهی است که داده های زیادی برای پیاده سازی دروس الکترونیک لازم است اما مجموعه ای از داده ها هستند که از اهمیت بیشتری برخوردارند که برخی از آنها در جدول ۱ قابل مشاهده است.

| توصیف                                                                                                 | هدف                                                            | فرا داده های شی یادگیری                                                                                                   |
|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| معیار اصلی انتخاب یک شی یادگیری از مخزن اشیاء یادگیری در دسترس.                                       | لیست اولیه ی یک شی یادگیری برای قرار گرفت در یک درس الکترونیک. | عنوان، توضیح، زبان، پلتفرم، کلمات کلیدی، مباحث پوشش داده شده، نوع، کیفیت ویژه، برنامه یادگیری، هدف، کپی رایت، محدودیت ها. |
| زمانیکه هیچ شی یادگیری با ویژگی های مد نظر ما وجود ندارد و یا وجود دارد اما مدل ارزیابی مناسبی ندارد. | برای تصمیم گیری در خصوص توسعه ی اشیا یادگیری جدید.             | نوع LOI (مطلب، توضیح، مثال)، هدف، موضوع، نوع LOT                                                                          |

<sup>۱۹</sup> e-learning course or ELC

<sup>۲۰</sup> learning object

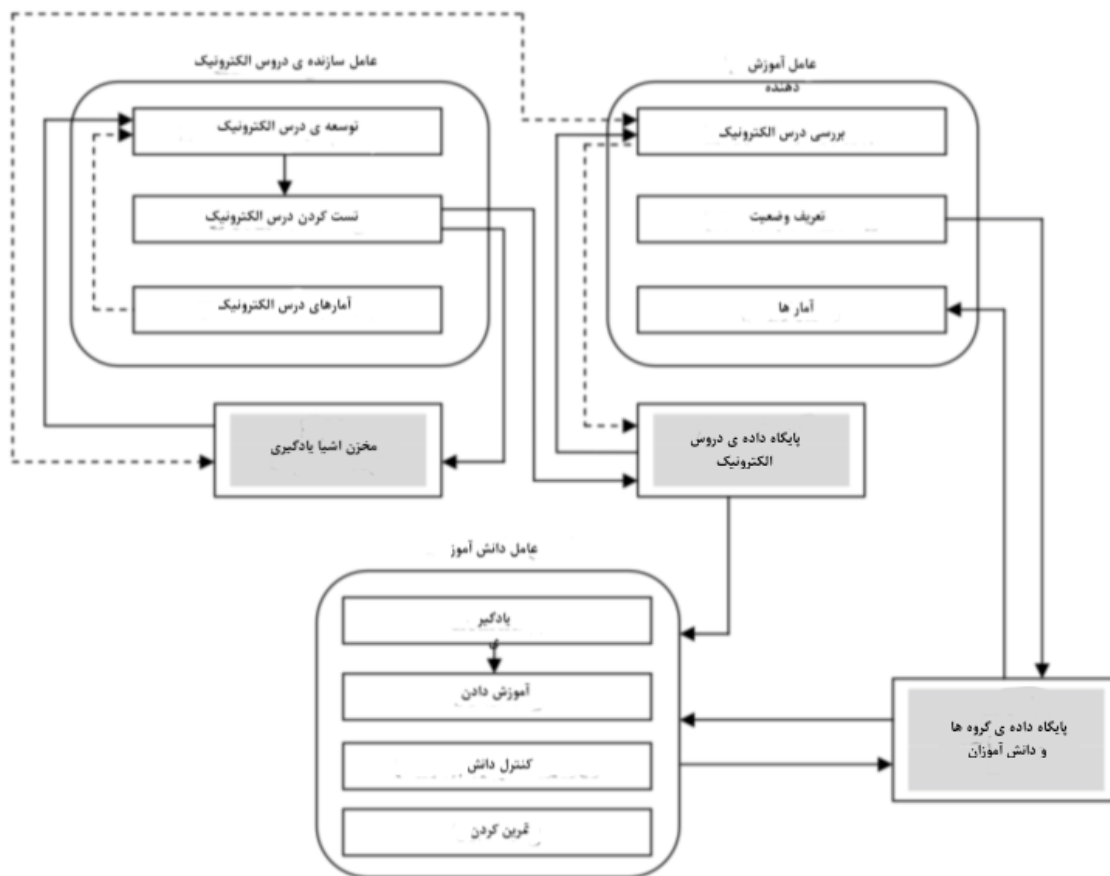
<sup>۲۱</sup> learning object information

<sup>۲۲</sup> learning object task

همانطور که پیش از این هم گفته شد اشیا یادگیری یکی از مولفه های درس های الکترونیک هستند که به همراه برنامه های پردازشی مختلف به کار گرفته میشوند. بنابراین، برای پیاده سازی راهبرد مبتنی بر مولفه در پیاده سازی سامانه های مدیریت آموزش مخزن های شی یادگیری و مولفه های برنامه های پردازشی ضروری هستند.

### معماری سامانه مدیریت آموزش

همانطور که پیش تر هم اشاره شد، انواع مختلفی از کاربران می توانند از سامانه ی مدیریت آموزش استفاده کنند مانند: دانش آموزان، آموزش دهندگان، سازندگان دروس الکترونیک (ELCA)<sup>۳</sup>، مدیران، اپراتورها و غیره. اما کاربران اصلی را میتوان همان سه مورد اول در نظر گرفت، ما هم در پیاده سازی معماری فقط سه عامل سازنده ی دروس الکترونیک، دانش آموز، آموزش دهنده را در نظر میگیریم. شکل ۱ زیر نشان دهنده ی معماری این سامانه است.



تصویر ۱. معماری سامانه ی مدیریت آموزش

هر سامانه ی مدیریت آموزش یک برنامه ی اصلی دارد که هسته ی سیستم است و وظایفی چون: شناسایی کاربران، اتصال آنها، فراهم آوردن رابط کاربری یکسان برای همه ی کاربران، سازماندهی و نگهداری نشست<sup>۲۴</sup>ها، نگهداری پایگاه داده ها و مخزن ها.

اشیا یادگیری و فراداده های آنها در مخزن اشیا یادگیری<sup>۲۵</sup> قرار میگیرند و میتوانند توسط سازنده ی دروس الکترونیک برای بهبود و توسعه ی یادگیری الکترونیک به کار گرفته شوند. دروسی که توسعه داده شده و تست هم شده اند در پایگاه داده ی دروس الکترونیک ذخیره میشوند. هر آموزش دهنده پس از بررسی دروس به دانش آموزان یک یا چند مورد را پیشنهاد میدهد. دانش آموزان میتوانند از آن دروس برای یادگیری، آموزش دادن، کنترل دانش و تمرین کردن استفاده کنند. پایگاه داده ی دانش آموزان و گروه ها اطلاعاتی را راجع به دانش آموزان و گروه های آنها شامل میشود. کارکرد های اصلی که توسط سازنده ی دروس الکترونیک (ELCA) ارائه میشوند، توسعه ی درس و تست کردن آن است. فرآیند توسعه ی یک درس الکترونیک بر اساس راهبرد مبتنی بر مولفه از پنج مرحله تشکیل شده است:

- ۱) تعریف اهداف و انگیزه های یک درس الکترونیکی.
  - ۲) توسعه ی سناریوی یک درس الکترونیک با اطلاعاتی درباره ی اشیا یادگیری و فرا داده های آنها.
  - ۳) انتخاب اشیا لازم یادگیری از مخزن اشیا یادگیری که در این مرحله یک تحلیل صورت میگیرد و اشیا یادگیری انتخاب میشوند که در جهت هدف یادگیری باشند.
  - ۴) تغییر اشیا انتخاب شده یا پیاده سازی اشیا جدید و تست آنها
  - ۵) جمع اشیا یادگیری در داخل یک درس الکترونیک با توجه به سناریوی مطرح شده در گام دوم. بعد از این مرحله، تست کلی سامانه صورت میگیرد و بعد در پایگاه داده ی دروس ذخیره میشود. کارکرد «آمار های درس های الکترونیک» به نویسنده اجازه میدهد که درباره ی دانش آموزانی که آن درس را اخذ کرده اند، اطلاعات جمع آوری کند و بعد از تحلیل آنها سازنده میتواند کیفیت دروس را با اعمال برخی تغییرات بهبود بخشد.
- عامل بعدی آموزش دهنده است؛ که کارکرد های زیر را دارد:

- بررسی دروس الکترونیک: پیش از ارائه ی یک درس الکترونیک به دانش آموزان، آموزش دهنده باید آن را مورد بررسی و ارزیابی قرار دهد تا ببیند آیا اهداف تدریس با درس همخوانی دارد یا خیر و همچنین تصمیم میگیرد که آیا آن را بدون هیچ تغییری ارائه دهد یا در آن تغییراتی ایجاد کند به عنوان مثال، وظایف بیشتری تعریف کند یا مثال های بیشتری بزند. در نهایت یک رونوشت از درس مد نظر با تغییرات آموزش دهنده در پایگاه داده ذخیره میشود.
- تعریف وضعیت: بسته به استراتژی انتخاب شده برای استفاده از درس الکترونیک، آموزش دهنده می تواند یک مجموعه پارامتر را برای نشان دادن آن درس تعریف کند. به عنوان مثال میتواند میزان سختی سطح های مختلف و سطح دسترسی شرکت کنندگان را تعیین کند.
- آمار ها: به آموزش دهنده اجازه میدهند اطلاعات دقیقی از روند کار دانش آموزان داشته باشد.

---

<sup>۲۴</sup> session

<sup>۲۵</sup> learning object repository



- عامل سوم و آخر نیز دانش آموز است که چهار کارکرد زیر را برایش متصوریم:
- یادگیری: اولین و بدیهی ترین کارکرد این عامل می باشد و به دانش آموز اجازه می دهد هر درسی را که در پایگاه داده است و به آن دسترسی دارد را به صورت مستقل اخذ کند.
  - آموزش دادن: این مسئولیت توسط آموزش دهنده بر عهده ی دانش آموز قرار میگیرد و با توجه به سناریوی تعیین شده توسط وی تبیین میگردد. مثال ملموس این کارکرد را میتوان دستیاران آموزشی دروس دانشگاه تعریف کرد.
  - کنترل یادگیری یا دانش: برای ارزیابی و امتحان دانش آموزان به کار گرفته میشود.
  - تمرین کردن: ارائه ی آمار و گزارش دهی به این صورت که در ازای هر عملکرد دانش آموز یک کامنت دقیق به آن اختصاص میدهد تا توانایی هایش را با توجه به آن گسترش دهد.[۳]

#### تکنولوژی های پیشنهادی برای سامانه ی مدیریت آموزش

با در نظر گرفتن پیچیدگی های معماری سیستم های مدیریت آموزش، این سامانه بر اساس یک ساختار چند لایه پیاده سازی میشود که برای بسیاری از سیستم های نرم افزار مدرن استاندارد است. این سامانه بر اساس تکنولوژی های مبتنی بر وب است چرا که این راه موثر ترین راه ارائه ی محتواست. سامانه های مدیریت آموزش در چند سال اخیر توسعه ی چشم گیری داشته اند و اجازه ی پردازش ساده ی همه ی انواع اطلاعات را که در فرآیند یادگیری الکترونیک به کار میروند را میدهند. در سمت کلاینت یک رابط کاربری مبتنی بر مرورگر داریم که تمام کارکرد های لازم برای نشان دادن مولفه های یادگیری الکترونیک برای کاربر را دارد. در حالیکه در سمت سرور همه ی منطق محاسباتی، پایگاه داده و مخازن را داریم.

#### تکنولوژی های سمت کلاینت

در سمت کاربر پیاده سازی با استفاده از استاندارد های فعلی تکنولوژی وب صورت میگیرد. Ajax، که اجازه میدهد که محتوا را به صورت پویا و وابسته به ورودی تغییر دهیم و کار پردازشی روی سرور را کاهش میدهد و همچنین نیازمندی های امنیتی را به طور کامل لحاظ میکند، یک تکنولوژی لازم است. برای طراحی رابط کاربری جذاب و کاربر پسند از framework هایی مانند jquery، dojo، mootools استفاده میکنیم. یکی از بهینه ترین framework های سمت سرور mootools است؛ که تضمین میکند رابط کاربری در همه ی مرورگر های موجود در بازار یکسان باشد.

#### تکنولوژی های سمت سرور

در سمت سرور ما منطق کار و سیستم های پایگاه داده را داریم. با در نظر گرفتن چرخه ی زندگی یک نرم افزار، پنج تا هفت سال، تکنولوژی های استفاده شده در پیاده سازی منطق کار باید قابل اطمینان باشند؛ به این معنی که پشتیبانی طولانی تری داشته باشند، تغییرات گسترده ای در syntax آنها وجود نداشته باشد و همچنین جامعه ی توسعه دهندگان گسترده ای داشته باشند مثل java، .NET، PHP. ما در سمت سرور این زبان را جاوا انتخاب میکنیم چراکه این زبان علاوه بر مزیت های بالا، محیط های توسعه ی یکپارچه ی خوب و همچنین framework های خوبی هم دارد. یکی از مهم ترین مزایای پیاده سازی با این تکنولوژی ها این است که اکثر آنها رایگان است و بنابراین هزینه های توسعه کاهش میابد. برای ذخیره سازی داده ها، آنها را در postgresql نگهداری میکنیم؛ چراکه رایگان و متن باز است و انعطاف پذیر است و ابزار قدرتمندی برای مدیریت داده ها داخل پایگاه داده دارد.[۴]

## برنامه ی تست نرم افزار

در اینجا test plan ای که برای این پروژه در نظر داریم را گام به گام توضیح می دهیم و در حین آن به بررسی unit test , environment testing , integration test می پردازیم.

### برنامه آزمون یا Test plan چیست؟

برنامه آزمون یک سند مفصل است که استراتژی تست ، اهداف ، برنامه ، ارزیابی و منابع مورد نیاز برای آزمایش را شرح می دهد. Test plan به ما کمک می کند تا تلاش لازم برای اعتبارسنجی کیفیت برنامه مورد آزمایش را تعیین کنیم. Test plan به عنوان یک طرح برای انجام فعالیت های تست نرم افزار به عنوان یک فرآیند تعریف شده است که توسط مدیر تست کنترل می شود. برای نوشتن Test plan ۸ قدم را باید طی کنیم:

۱. Analyze the product
۲. Design the Test Strategy
۳. Define the Test Objectives
۴. Define Test Criteria
۵. Resource Planning
۶. Plan Test Environment
۷. Schedule & Estimation
۸. Determine Test Deliverables

قدم اول: آیا می توان بدون هیچ گونه اطلاعاتی در مورد آن ، محصولی را آزمایش کرد؟ جواب منفی است. شما باید قبل از آزمایش ، محصول را کاملاً شناسایی کنید مثلاً:

چه کسی از وب سایت استفاده می کند؟  
وبسایت برای چه کاری استفاده می شود؟  
چگونه کار خواهد کرد؟

نرم افزار / سخت افزار مورد استفاده محصول چیست؟

در پروژه مدنظر ما کارمندان و مدیران شرکت از این وب سایت استفاده می کنند و معماری این نرم افزار نیز در بخش قبل توصیف شده است.

برای تحلیل سایت می توان از روش های زیر استفاده کنید:

- استفاده از نرم افزار
- مصاحبه با developer و designer
- بررسی مستندات پروژه

قدم دوم: استراتژی آزمون یک گام مهم در تهیه یک برنامه آزمون است. یک سند استراتژی Test ، یک سند سطح بالا است که معمولاً توسط Test Manager تهیه می شود. این سند تعریف می کند:

۱. اهداف آزمایش پروژه و ابزار دستیابی به آنها
۲. تلاش و هزینه آزمایش را تعیین می کند.

این قدم به تنهایی شامل ۴ بخش است که توضیح داده می شود:

بخش اول: شناسایی scope ها

قبل از شروع هرگونه فعالیت آزمایشی باید دامنه آزمایش مشخص شود. اجزای سیستم مورد آزمایش (سخت افزار ، نرم افزار ، واسطه و غیره) به عنوان "در دامنه" تعریف می شوند. اجزای سیستم که

آزمایش نخواهد شد نیز باید به وضوح به عنوان "خارج از محدوده" تعریف شوند. تعیین دامنه پروژه برای همه ذینفعان بسیار مهم است. دامنه دقیق به ما کمک می کند:

۱. اطلاعات دقیق از آزمایشاتی که انجام می دهیم به همه بدهد.
۲. تمام اعضای پروژه درک واضحی از آنچه تست شده است دارند.

حال دامنه پروژه خود را چگونه تعیین کنیم؟ برای تعیین دامنه ، به موارد زیر توجه می کنیم:

۱. نیاز مشتری
۲. بودجه پروژه
۳. مشخصات محصول
۴. مهارت و استعداد تیم آزمون

بخش دوم: تشخیص testing type

Testing Type یک روش تست استاندارد است که نتیجه آزمون مورد انتظار را مشخص میکند. هر نوع آزمایش برای شناسایی نوع خاصی از اشکالات محصول فرموله شده است اما ، تمام انواع تست با هدف دستیابی به یک هدف مشترک "کشف زود هنگام همه نقص ها قبل از عرضه محصول به مشتری" انجام می شود.

انواع تست های متداول مانند شکل زیر شرح داده شده است:

|                           |                                                                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit Test                 | • Test the <b>smallest</b> piece of <b>verifiable</b> software in the application                                                              |
| API Testing               | • <b>Test</b> the <b>API's</b> created for the application                                                                                     |
| Integration Test          | • Individual software modules are <b>combined</b> and tested as a group                                                                        |
| System Test               | • Conducted on a <b>complete, integrated</b> system to evaluate the system's compliance with its specified requirements                        |
| Install/uninstall Testing | • Focuses on what <b>customers</b> will <b>need</b> to do to <b>install</b> / <b>uninstall</b> and set up/remove the new software successfully |
| Agile Testing             | • Testing the system using Agile methodology                                                                                                   |

Unit test و integration تست دو موردی هستند که در انتها بیشتر روی آن صحبت می کنیم.

بخش سوم: document risk

ریسک یک رویداد نامشخص در آینده است که احتمال وقوع آن و احتمال از دست دادن وجود دارد. وقتی ریسک واقع میشود ، تبدیل به "مسئله" می شود.

بخش چهارم: ساخت test logistics

در Test Logistics ، مدیر آزمون باید به سؤالات زیر پاسخ دهد:

۱. چه کسی تست خواهد کرد؟

۲. چه موقع آزمایش انجام می شود؟

قدم سوم: هدف از این آزمایش یافتن هرچه بیشتر نقص نرم افزار است. قبل از انتشار ، اطمینان حاصل کنیم که نرم افزار مورد آزمایش بدون اشکال است.

برای تعریف اهداف آزمون ، باید ۲ مرحله زیر را انجام دهیم:

۱. تهیه لیست تمام ویژگی های نرم افزار (عملکرد ، رابط کاربری گرافیکی ...) که ممکن است نیاز به

آزمایش داشته باشند.

۲. هدف یا اهداف آزمون را بر اساس ویژگی های فوق تعریف کنید  
این مراحل را برای یافتن هدف آزمون پروژه خود اعمال می کنیم.

می توانید از روش "TOP-DOWN" برای یافتن ویژگی های وب سایت که ممکن است نیاز به آزمایش داشته باشد، استفاده کنیم.

در این روش ، برنامه مورد آزمایش را به جزء و زیر کامپوننت تقسیم می کنیم. در مبحث قبلی ، ما قبلاً مشخصات مورد نیاز را تجزیه و تحلیل کرده ایم ، بنابراین می توانیم یک Mind-Map ایجاد کنیم تا ویژگی های وب سایت را پیدا کنیم.

قدم چهارم: معیار های آزمون را طراحی کنید.

معیارهای آزمون یک استاندارد یا قانونی است که بر اساس آن می توان رویه آزمون را پایه گذاری کرد. معیارهای تعلیق (suspension criteria)

معیارهای مهم تعلیق برای یک آزمون را مشخص می کنیم. اگر معیارهای تعلیق در طول آزمایش رعایت شود ، چرخه تست فعال تا زمانی که معیارها بررسی شود متوقف می شود.

مثال: اگر اعضای تیم گزارش دادند که ۴۰٪ از موارد آزمایش شکست خورده است ، باید آزمایش را به حالت تعلیق در بیاوریم تا اینکه تیم توسعه تمام موارد شکست خورده را برطرف کند.

قدم پنجم: برنامه ریزی منابع

برنامه منابع خلاصه ای از انواع منابع مورد نیاز برای انجام کار پروژه است. منابع می توانند انسانی ، تجهیزات و مواد مورد نیاز برای انجام یک پروژه باشند.

برنامه ریزی منابع، عامل مهمی برای برنامه ریزی آزمون است زیرا به تعیین تعداد منابع (کارمند ، تجهیزات و ...) برای پروژه کمک می کند. بنابراین، مدیر آزمون می تواند برنامه و برآورد صحیحی را برای پروژه انجام دهد.

این بخش منابع پیشنهادی برای پروژه ما را نشان می دهد.

منابع انسانی : test manager – tester – developer

منابع سیستم: computer – network – server – test tool

قدم ششم: برنامه ریزی برای test environment

یک محیط تست مجموعه نرم افزاری و سخت افزاری است که تیم آزمایش بر روی آن تست انجام می دهد. محیط آزمایش شامل محیط واقعی کسب و کار و کاربر و همچنین محیطهای فیزیکی مانند سرور ، محیط در حال اجرا است.

برای اتمام این کار ، به همکاری محکم بین تیم تست و تیم توسعه نیاز داریم.

برای درک واضح برنامه وب تحت آزمایش باید از توسعه دهنده سؤال کنیم. در اینجا برخی از سوالات آورده شده است. البته در صورت نیاز می توانیم سؤالات دیگر را نیز بپرسیم.

۱. حداکثر اتصال کاربر که این وب سایت می تواند همزمان انجام دهد چیست؟

۲. الزامات سخت افزاری / نرم افزاری برای نصب این وب سایت چیست؟

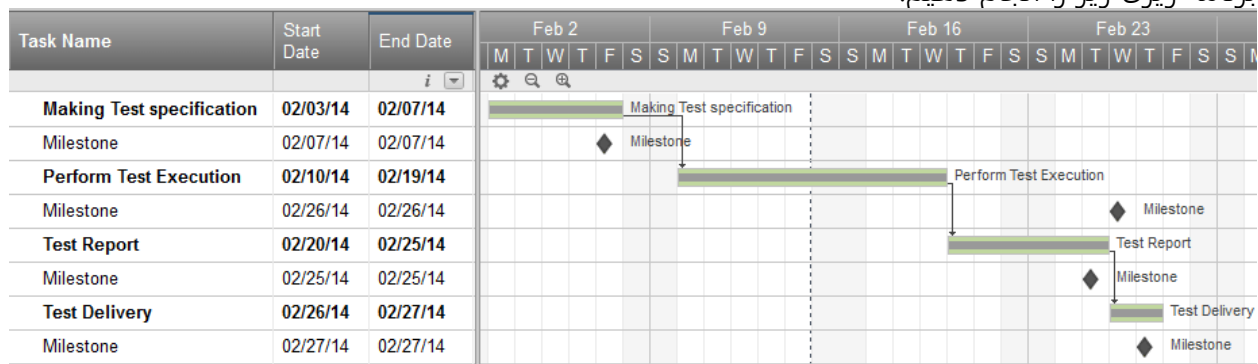
۳. آیا کامپیوتر کاربر برای مرور وب سایت به تنظیم خاصی نیاز دارد؟

قدم هفتم: برنامه و تخمین

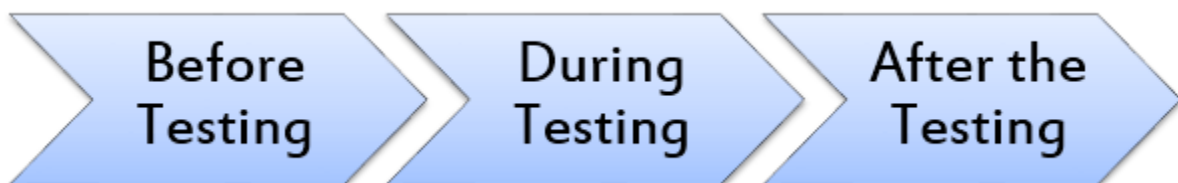
در مرحله Test Estimation ، فرض کنید کل پروژه را به کارهای کوچک تقسیم کرده و تخمین را برای هر کار به شرح زیر اضافه کرده ایم.

| Task                          | Members                    | Estimate effort |
|-------------------------------|----------------------------|-----------------|
| Create the test specification | Test Designer              | 170 man-hour    |
| Perform Test Execution        | Tester, Test Administrator | 80 man-hour     |
| Test Report                   | Tester                     | 10 man-hour     |
| Test Delivery                 |                            | 20 man-hour     |
| Total                         |                            | 280 man-hour    |

حال برنامه ای را برای انجام این کارها ایجاد می کنیم.  
 برنامه ریزی یک اصطلاح معمول در مدیریت پروژه است. مدیر برنامه با ایجاد یک برنامه در برنامه ریزی آزمون ، می تواند از آن به عنوان ابزاری برای نظارت بر پیشرفت پروژه استفاده کند، و بازپرداخت هزینه ها را نیز کنترل نماید.  
 فرض کنیم می خواهیم وب سایت مدیریت سامانه آموزش را در یک ماه راه اندازی کنیم، می توانیم برنامه ریزی زیر را انجام دهیم:



قدم هشتم: تحویل تست  
 Test Deliverables لیستی از کلیه اسناد ، ابزارها و سایر مؤلفه هایی است که برای حمایت از تلاش برای آزمایش باید تهیه و نگهداری شود.  
 در هر مرحله از چرخه توسعه نرم افزار تحویل های مختلف تست وجود دارد.



قبل از تست:

- Test plans document.
- Test cases documents
- Test Design specifications.

در حین تست:

- Test Scripts
- Simulators.
- Test Data
- Test Traceability Matrix
- Error logs and execution logs.

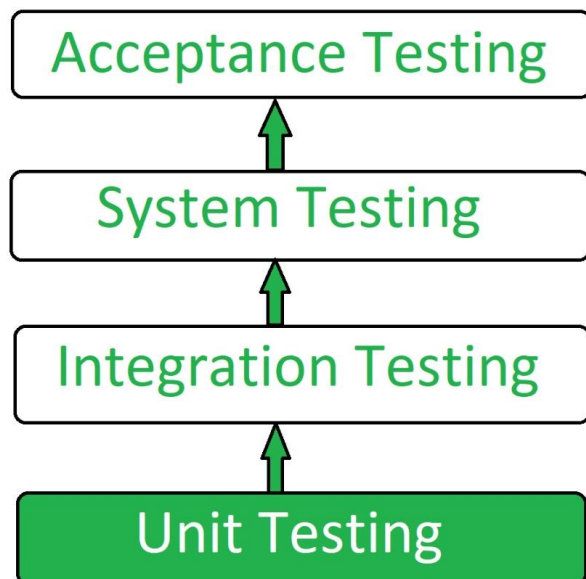
بعد از تست:

- Test Results/reports
- Defect Report
- Installation/ Test procedures guidelines
- Release notes

حال که test plan پروژه مشخص شد توضیح بیشتری در ارتباط با unit test و integration test می دهیم و اینکه این دو را باید به چه صورت انجام داد:

Unit test شامل شکستن برنامه به قطعه ها و انجام تست روی هر قطعه است. معمولاً تست ها به صورت برنامه های جداگانه اجرا می شوند، اما روش آزمایش با توجه به زبان و نوع نرم افزار مثل GUI ، خط فرمان ، کتابخانه متفاوت است. در اکثر زبانها چارچوب های تست واحد وجود دارد ، ما با توجه به معماری سیستم خودمان باید این کار را انجام دهیم. معمولاً پس از هر تغییر کد منبع ، آزمایشها بصورت دوره ای انجام می شود. هرچه زودتر بهتر، چون می توان مشکل را زودتر پیدا کرد و آنرا رفع کرد.

Integration test فرآیند آزمایش رابط بین دو واحد نرم افزاری یا ماژول است. تمرکز آن بر تعیین صحت رابط است. هدف از این آزمون ، افشای خطاها در تعامل بین واحدهای یکپارچه است. هنگامی که همه ماژول ها تست واحد شده اند ، تست ادغام انجام می شود.



برای ساخت unit test قدم به قدم به این صورت عمل می کنیم:

۱. یک ابزار / چارچوبی برای زبان خود پیدا کنید.
۲. موارد آزمایش را برای همه چیز ایجاد نکنید. در عوض ، روی تست هایی که بر رفتار سیستم تأثیر می گذارد ، تمرکز کنید.
۳. محیط توسعه را از محیط آزمایش جدا کنید.
۴. از داده های آزمایشی استفاده کنید که نزدیک به تولید باشد.
۵. قبل از رفع نقص ، آزمایشی بنویسید که نقص آن را در معرض دید شما قرار دهد.
۶. موارد آزمون را بنویسید که مستقل از یکدیگر باشند. به عنوان مثال ، اگر یک کلاس به یک پایگاه داده بستگی دارد ، یک مورد را بنویسید که با بانک اطلاعاتی در تعامل باشد تا کلاس را آزمایش کند. در عوض ، یک رابط انتزاعی در اطراف آن اتصال بانک اطلاعاتی ایجاد کنید و آن رابط را با یک شیء پیاده سازی کنید.
۷. اطمینان حاصل کنید که برای پیگیری اسکرپت های تست خود از یک سیستم کنترل نسخه استفاده می کنید.
۸. علاوه بر نوشتن موارد برای تأیید رفتار ، مواردی را بنویسید تا از عملکرد کد اطمینان حاصل شود.
۹. تست های واحد را بطور مداوم و مکرر انجام دهید.

برای ساخت integration test نیز به این صورت عمل می کنیم:

۱. اطمینان حاصل کنید که یک سند طراحی جزئیات مناسب را در جایی دارید که تعامل بین هر واحد به روشنی تعریف شود. در واقع ، شما قادر به انجام آزمایش یکپارچه سازی بدون این اطلاعات نخواهید بود.
۲. اطمینان حاصل کنید که شما یک سیستم مدیریت پیکربندی نرم افزار قوی دارید. در غیر اینصورت ، شما زمان سختی را برای ردیابی نسخه مناسب هر واحد خواهید داشت ، به خصوص اگر

تعداد واحدهای یکپارچه شده بسیار زیاد باشد.  
۳. قبل از شروع آزمون ادغام اطمینان حاصل کنید که هر واحد ، تست واحد شده است.  
۴. تا حد امکان تست های خود را بطور خودکار انجام دهید ، به خصوص هنگامی که از روش Top Down یا Bottom Up استفاده می کنید ، زیرا تست رگرسیون دستی می تواند ناکارآمد باشد.

## منابع

<https://app.moqups.com/>

<https://files.eric.ed.gov/fulltext/ED067308.pdf>

[http://www.calhr.ca.gov/Documents/LMS%20Functional%20and%20Non Functional%20Requirements.docx](http://www.calhr.ca.gov/Documents/LMS%20Functional%20and%20Non%20Functional%20Requirements.docx)

[http://etutorials.org/Programming/Software+architecture+in+practice,+second+edition/Part+Two+Creating+an+Architecture/Chapter+0.+Achieving+Qualities/0.1+Introducing+Tactics/](http://etutorials.org/Programming/Software+architecture+in+practice,+second+edition/Part+Two+Creating+an+Architecture/Chapter+0.+Achieving+Qualities/0.1+Introducing+ Tactics/)

<https://www.guru99.com/what-everybody-ought-to-know-about-test-planing.html>

<https://www.geeksforgeeks.org/unit-testing-software-testing/>

<http://softwaretestingfundamentals.com/unit-testing/>

<http://softwaretestingfundamentals.com/integration-testing/>